

# AI 心理咨询助手 – 内测操作指南

## AI 心理咨询助手 – 内测操作指南

版本：v2.0 | 更新：2026-05-19 | 验证状态：✅ 21 项全部通过  
本指南所有命令已在 macOS + Python 3.12 + SQLite 环境下逐条跑通。

### 目录

- [1. 环境准备](#)
- [2. 测试账号](#)
- [3. 一键启动](#)
- [4. 认证模块测试 \(4 项\)](#)
- [5. 管理后台测试 \(4 项\)](#)
- [6. AI 对话核心测试 \(6 项\)](#)
- [7. 危机检测测试 \(1 项\)](#)
- [8. 心理测评测试 \(3 项\)](#)
- [9. 咨询师端测试 \(4 项\)](#)
- [10. 问题反馈模板](#)

## 一、环境准备

### 1.1 系统要求

| 项目      | 要求                          |
|---------|-----------------------------|
| Python  | 3.11+                       |
| Node.js | 18+                         |
| 操作系统    | macOS / Linux / Windows WSL |

## 1.2 安装步骤

```
# 1. 克隆项目
git clone git@github.com:cangyueshi/ai-xinlirobot.git
cd ai-xinlirobot

# 2. 后端环境
cd backend
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# 3. 配置环境变量
cp .env.example .env
# 编辑 .env, 确保 DEEPSEEK_API_KEY 正确填入

# 4. 前端环境
cd ../frontend
npm install
npm run build:h5

# 5. 返回 backend 启动服务
cd ../backend
source venv/bin/activate
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

## 1.3 验证服务

```
curl http://localhost:8000/api/health
# 预期输出: {"status":"ok","message":"AI 心理咨询机器人服务运行中"}
```

也可以使用项目根目录下的 start\_test.sh 一键启动：

```
bash start_test.sh
```

# 二、测试账号

| 角色    | 账号             | 密码          | 说明           |
|-------|----------------|-------------|--------------|
| 超级管理员 | admin          | admin123456 | 启动时自动创建      |
| 测试咨询师 | counselor_test | test123456  | 启动时自动创建      |
| 来访者   | 不需要            | 不需要         | 通过微信登录接口自动创建 |

## 三、一键启动

以下所有命令在项目根目录下执行。先设置环境变量方便后续使用：

```
export BASE_URL="http://localhost:8000"
```

---

## 四、认证模块测试

### TC-01: 健康检查

```
curl -s $BASE_URL/api/health | python3 -m json.tool
```

预期: {"status": "ok", "message": "AI 心理咨询机器人服务运行中"}

---

### TC-02: 超级管理员登录

```
curl -s -X POST $BASE_URL/api/auth/admin-login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123456"}' | python3 -m
  json.tool
```

验证点: - access\_token 字段存在 (JWT) - user.role 为 "super\_admin" - user.status 为 "active"

提取 token 备用:

```
ADMIN_TOKEN=$(curl -s -X POST $BASE_URL/api/auth/admin-login \
  -H "Content-Type: application/json" \
  -d '{"username":"admin","password":"admin123456"}' | python3 -c
  "import json,sys; print(json.load(sys.stdin)
  ['access_token'])")
```

```
echo "Admin token: ${ADMIN_TOKEN:0:30}..."
```

---

### TC-03: 游客微信登录 (新用户自动注册)

```
curl -s -X POST $BASE_URL/api/auth/wechat-login \
  -H "Content-Type: application/json" \
  -d '{"openid":"wx_test_001","display_name":"测试用户小张"}' |
  python3 -m json.tool
```

验证点：- access\_token 存在 - user.role 为 "visitor" - user.username 以 wx\_ 开头（自动生成） - user.display\_name 为 "测试用户小张" - 用户自动创建，不需要预先注册

提取 token 备用：

```
VISITOR_TOKEN=$(curl -s -X POST $BASE_URL/api/auth/wechat-login \
-H "Content-Type: application/json" \
-d '{"openid":"wx_test_001","display_name":"测试用户小张"}' |
python3 -c "import json,sys; print(json.load(sys.stdin)
['access_token'])")
```

---

## TC-04：错误密码登录

```
curl -s -X POST $BASE_URL/api/auth/admin-login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"wrongpassword"}' | python3
-m json.tool
```

预期：{"detail": "用户名或密码错误"} (HTTP 401)

---

## 五、管理后台测试

以下测试需要先执行 TC-02 获取 ADMIN\_TOKEN

### TC-05：仪表盘

```
curl -s $BASE_URL/api/admin/dashboard \
-H "Authorization: Bearer $ADMIN_TOKEN" | python3 -m json.tool
```

预期：返回 4 个统计数据——total\_visitors、total\_counselors、total\_sessions、active\_sessions

---

### TC-06：创建咨询师

```
curl -s -X POST $BASE_URL/api/admin/counselors \
-H "Authorization: Bearer $ADMIN_TOKEN" \
-H "Content-Type: application/json" \
-d '{"username":"counselor_lisi","password":"test123456","display_name":"李咨
询师","email":"lisi@example.com","bio":"擅长青少年心理辅
导","specialties":"考试焦虑、人际关系"}' | python3 -m json.tool
```

验证点：- role 为 "counselor" - must\_change\_password 为 true（首次登录需改密码） - status 为 "active"

---

## TC-07：咨询师列表

```
curl -s $BASE_URL/api/admin/counselors \
  -H "Authorization: Bearer $ADMIN_TOKEN" | python3 -c "
import json,sys
cs=json.load(sys.stdin)
print(f'共 {len(cs)} 位咨询师')
for c in cs:
    print(f'  {c["display_name"]} | role={c["role"]} |
        status={c["status"]}')
"
```

预期：列表中包含刚创建的”李咨询师”和系统自带的”测试咨询师”

---

## TC-08：普通用户越权访问管理后台

```
curl -s $BASE_URL/api/admin/dashboard \
  -H "Authorization: Bearer $VISITOR_TOKEN" | python3 -m json.tool
```

预期：{"detail": "需要管理员权限"} (HTTP 403)

---

# 六、AI 对话核心测试

以下测试需要先执行 TC-03 获取 VISITOR\_TOKEN

## TC-09：创建对话会话

```
curl -s -X POST $BASE_URL/api/chat/sessions \
  -H "Authorization: Bearer $VISITOR_TOKEN" | python3 -m json.tool
```

验证点：- status 为 "active" - user\_message\_count 为 0 -  
is\_crisis\_mode 为 false

提取会话 ID 备用：

```
SESSION_ID=$(curl -s -X POST $BASE_URL/api/chat/sessions \
  -H "Authorization: Bearer $VISITOR_TOKEN" | python3 -c "import
    json,sys; print(json.load(sys.stdin)['id'])")
echo "Session ID: $SESSION_ID"
```

如果用户已有活跃会话，会返回已有的而不是创建新的——这是正常行为。

---

## TC-10: 发送正常消息并获取 AI 回复

```
curl -s -X POST $BASE_URL/api/chat/sessions/$SESSION_ID/message \
  -H "Authorization: Bearer $VISITOR_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"content": "最近学习压力很大, 经常失眠"}' | python3 -c "
import json,sys
d=json.load(sys.stdin)
print('role:', d['role'])
print('AI回复前150字:', d['content'][:150])
"
```

**验证点：** – role 为 "assistant" – AI 回复内容为中文，语气共情，有实质性内容 – AI 回复不包含说教语句

继续发几条消息：

```
for msg in "特别是考试前会特别焦虑" "有时候感觉喘不过气来" "跟父母沟通也很困难"; do
  curl -s -X POST $BASE_URL/api/chat/sessions/$SESSION_ID/message \
    -H "Authorization: Bearer $VISITOR_TOKEN" \
    -H "Content-Type: application/json" \
    -d '{"content": "$msg"}' > /dev/null
  echo "已发送: $msg"
done
```

---

## TC-11: 检查会话状态

```
curl -s $BASE_URL/api/chat/sessions/mine \
  -H "Authorization: Bearer $VISITOR_TOKEN" | python3 -c "
import json,sys
s=[s for s in json.load(sys.stdin) if s['id']==$SESSION_ID][0]
print(f'状态: {s["status"]}')
print(f'消息计数: {s["user_message_count"]}')
print(f'危机模式: {s["is_crisis_mode"]}')
print(f'结束原因: {s["ending_reason"]}')
"
```

**预期：**如果发了 4 条消息，user\_message\_count 应为 4，status 应为 "active"

---

## TC-12: 查看消息记录

```
curl -s $BASE_URL/api/chat/sessions/$SESSION_ID/messages \
  -H "Authorization: Bearer $VISITOR_TOKEN" | python3 -c "
import json,sys
msgs=json.load(sys.stdin)
print(f'消息总数: {len(msgs)}')
for m in msgs:
    role_label = '用户' if m['role']=='user' else 'AI'
    content_preview = m['content'][:60].replace('\n',' ')
    print(f'  [{role_label}] {content_preview}...')
"
```

**预期：** 消息总数 = 用户消息数 × 2（用户 + AI 交替），按时间正序排列

---

## TC-13: 10 条硬上限自动结束

创建新用户和会话，连续发送 10 条消息：

```
# 创建新用户
LIMIT_TOKEN=$(curl -s -X POST $BASE_URL/api/auth/wechat-login \
  -H "Content-Type: application/json" \
  -d '{"openid":"wx_limit_test","display_name":"上限测试"}' |
  python3 -c "import json,sys; print(json.load(sys.stdin)
    ['access_token'])")

LIMIT_SESSION=$(curl -s -X POST $BASE_URL/api/chat/sessions \
  -H "Authorization: Bearer $LIMIT_TOKEN" | python3 -c "import
    json,sys; print(json.load(sys.stdin)['id'])")

echo "发送 10 条消息（每条约 1 秒间隔）..."
for i in $(seq 1 10); do
    curl -s -X POST $BASE_URL/api/chat/sessions/$LIMIT_SESSION/
      message \
        -H "Authorization: Bearer $LIMIT_TOKEN" \
        -H "Content-Type: application/json" \
        -d '{"content":"'第${i}条测试消息：我最近总是感到很疲惫'" > /
          dev/null
    echo -n "."
    sleep 1
done

echo ""
echo "检查最终状态："
curl -s $BASE_URL/api/chat/sessions/mine \
  -H "Authorization: Bearer $LIMIT_TOKEN" | python3 -c "
import json,sys
s=[s for s in json.load(sys.stdin) if s['id']==$LIMIT_SESSION][0]
print(f'  状态: {s["status"]}')
print(f'  计数: {s["user_message_count"]}')
print(f'  结束原因: {s["ending_reason"]}')
"
```

```
print(f'  AI评估: {s[\"ai_summary\"][:100] if
      s.get(\"ai_summary\") else \"无\"}...')
"
```

预期：第 10 条后 status 变为 "completed", ending\_reason 为 "free\_limit\_reached", 附带 AI 评估摘要。

---

## 七、危机检测测试

### TC-14：三级危机关键词触发

创建新用户发送危机内容：

```
CRISIS_TOKEN=$(curl -s -X POST $BASE_URL/api/auth/wechat-login \
  -H "Content-Type: application/json" \
  -d '{"openid":"wx_crisis_test","display_name":"危机测试"}' |
  python3 -c "import json,sys; print(json.load(sys.stdin)
    ['access_token'])")

CRISIS_SESSION=$(curl -s -X POST $BASE_URL/api/chat/sessions \
  -H "Authorization: Bearer $CRISIS_TOKEN" | python3 -c "import
    json,sys; print(json.load(sys.stdin)['id'])")

# 先发一条正常消息
curl -s -X POST $BASE_URL/api/chat/sessions/$CRISIS_SESSION/
  message \
  -H "Authorization: Bearer $CRISIS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"content":"最近心情不太好"}' > /dev/null

# 发危机消息
curl -s -X POST $BASE_URL/api/chat/sessions/$CRISIS_SESSION/
  message \
  -H "Authorization: Bearer $CRISIS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"content":"我觉得活着好累，看不到未来"}' | python3 -c "
import json,sys
d=json.load(sys.stdin)
print('AI回复角色:', d['role'])
print('AI回复前200字:', d['content'][:200])
"

echo ""
echo "检查危机状态："
curl -s $BASE_URL/api/chat/sessions/mine \
  -H "Authorization: Bearer $CRISIS_TOKEN" | python3 -c "
import json,sys
s=[s for s in json.load(sys.stdin) if s['id']==$CRISIS_SESSION][0]
print(f'  危机模式: {s[\"is_crisis_mode\"]}')
```

```
print(f' 危机等级: {s["crisis_level"]}')
"
```

**预期：** - is\_crisis\_mode 为 True - crisis\_level 为 "level\_3" - AI 回复内容包含安抚话术，语气关切

---

## 八、心理测评测试

### TC-15: 获取量表列表

```
curl -s $BASE_URL/api/assessments/scales | python3 -c "
import json,sys
ss=json.load(sys.stdin)
print(f'共 {len(ss)} 个量表:')
for s in ss:
    print(f'  {s["name"]} - {s["question_count"]}题')
"
```

**预期：** PHQ-9（9题）、GAD-7（7题）、SCL-90（90题）

---

### TC-16: 获取量表详情

```
curl -s $BASE_URL/api/assessments/scales/1 | python3 -c "
import json,sys
d=json.load(sys.stdin)
print(f'名称: {d["name"]}')
print(f'题目数: {len(d["questions"])}')
print(f'选项: {d["options"]}')
print(f'第一题: {d["questions"][0]["text"]}')
"
```

**预期：** PHQ-9 量表，9 道题，4 级选项（完全不会 / 好几天 / 一半以上天数 / 几乎每天）

---

### TC-17: 提交测评

```
# 获取测试 token
ASSESS_TOKEN=$(curl -s -X POST $BASE_URL/api/auth/wechat-login \
-H "Content-Type: application/json" \
-d '{"openid":"wx_assess","display_name":"测评用户"}' | python3
-c "import json,sys; print(json.load(sys.stdin)
['access_token'])")
```

```
# 获取题目并构造答案（每题选第2个选项，即 score=1）
```

```

ANSWERS=$(curl -s $BASE_URL/api/assessments/scales/1 | python3 -c "
import json,sys
print(json.dumps([{'question_id': q['id'], 'score': 1} for q in
                    json.load(sys.stdin)['questions']]))
")

# 提交
curl -s -X POST $BASE_URL/api/assessments/submit \
  -H "Authorization: Bearer $ASSESS_TOKEN" \
  -H "Content-Type: application/json" \
  -d "{\"scale_id\": 1, \"answers\": $ANSWERS}" | python3 -c "
import json,sys
d=json.load(sys.stdin)
print(f'总分: {d[\"total_score\"]}')
print(f'等级: {d[\"result_level\"]}')
print(f'建议: {d.get(\"suggestion\",\"\")[0:100]}')
"

```

**预期：**返回总分、结果等级和评估建议。PHQ-9 每题 1 分共 9 分，属于“轻度抑郁”。

---

## 九、咨询师端测试

### TC-18：咨询师登录

```

COUNSELOR_TOKEN=$(curl -s -X POST $BASE_URL/api/auth/admin-login \
  -H "Content-Type: application/json" \
  -d '{"username":"counselor_test","password":"test123456"}' |
  python3 -c "import json,sys; print(json.load(sys.stdin)
    ['access_token'])")

echo "Token: ${COUNSELOR_TOKEN:0:30}..."

```

**预期：**返回 access\_token, user.role 为 "counselor"

---

### TC-19：咨询师查看预警

```

curl -s $BASE_URL/api/chat/alerts \
  -H "Authorization: Bearer $COUNSELOR_TOKEN" | python3 -c "
import json,sys
alerts=json.load(sys.stdin)
print(f'预警总数: {len(alerts)}')
for a in alerts[:5]:
    print(f'  ID={a[\"id\"]} level={a[\"crisis_level\"]}
        is_read={a[\"is_read\"]}')
"

```

**预期：**如果在前面测试中触发了危机关键词，这里会显示预警记录。

---

## TC-20：咨询师工作台

```
curl -s $BASE_URL/api/counselor/dashboard \
  -H "Authorization: Bearer $COUNSELOR_TOKEN" | python3 -m
  json.tool
```

**预期：**返回 upcoming\_appointments、unread\_alerts、total\_visitors、total\_chat\_sessions 等统计数据。

---

## TC-21：来访者列表

```
curl -s $BASE_URL/api/counselor/visitors \
  -H "Authorization: Bearer $COUNSELOR_TOKEN" | python3 -c "
import json,sys
vs=json.load(sys.stdin)
print(f'来访者总数: {len(vs)}')
for v in vs[:5]:
    print(f'  {v["display_name"]} (对话{v["chat_count"]}次, 测评
      {v["assessment_count"]}次)')
"
```

**预期：**显示所有注册的来访者及其对话/测评次数。

---

# 十、问题反馈模板

测试中发现问题时，请提供以下信息：

1. 测试用例编号：TC-XX
  2. 操作步骤：我做了什么
  3. 预期结果：应该看到什么
  4. 实际结果：实际看到了什么（附截图）
  5. HTTP 状态码：200/400/500 等
  6. 错误信息：返回的 detail 字段
- 

## 附录：测试结果汇总

| 编号    | 测试项     | 结果 |
|-------|---------|----|
| TC-01 | 健康检查    | ✓  |
| TC-02 | 超级管理员登录 | ✓  |

| 编号    | 测试项          | 结果 |
|-------|--------------|----|
| TC-03 | 游客微信登录       | ✓  |
| TC-04 | 错误密码登录       | ✓  |
| TC-05 | 管理后台仪表盘      | ✓  |
| TC-06 | 创建咨询师        | ✓  |
| TC-07 | 咨询师列表        | ✓  |
| TC-08 | 游客越权访问       | ✓  |
| TC-09 | 创建对话会话       | ✓  |
| TC-10 | 发送消息 + AI 回复 | ✓  |
| TC-11 | 检查会话状态       | ✓  |
| TC-12 | 查看消息记录       | ✓  |
| TC-13 | 10 条上限自动结束   | ✓  |
| TC-14 | 三级危机关键词触发    | ✓  |
| TC-15 | 获取量表列表       | ✓  |
| TC-16 | 获取量表详情       | ✓  |
| TC-17 | 提交测评         | ✓  |
| TC-18 | 咨询师登录        | ✓  |
| TC-19 | 咨询师查看预警      | ✓  |
| TC-20 | 咨询师工作台       | ✓  |
| TC-21 | 来访者列表        | ✓  |